

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
“КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені Ігоря Сікорського”
НАВЧАЛЬНО-НАУКОВИЙ ФІЗИКО-ТЕХНІЧНИЙ ІНСТИТУТ
Кафедра інформаційної безпеки

Лабораторна робота №3
з дисципліни «Технологія блокчейн та розподілені
системи»

Тема: Дослідження безпечної реалізації та експлуатації децентралізованих додатків.

Мета роботи: отримання навичок роботи із децентралізованими додатками та оцінка безпеки інформації при їх функціонуванні

Завдання: дослідження вимог OWASP (безпека web-додатків) та складання аналогічних вимог для обраної системи децентралізованих додатків.

Виконав: ФБ-51мн Тимощук Ілья

Київ 2026 р.

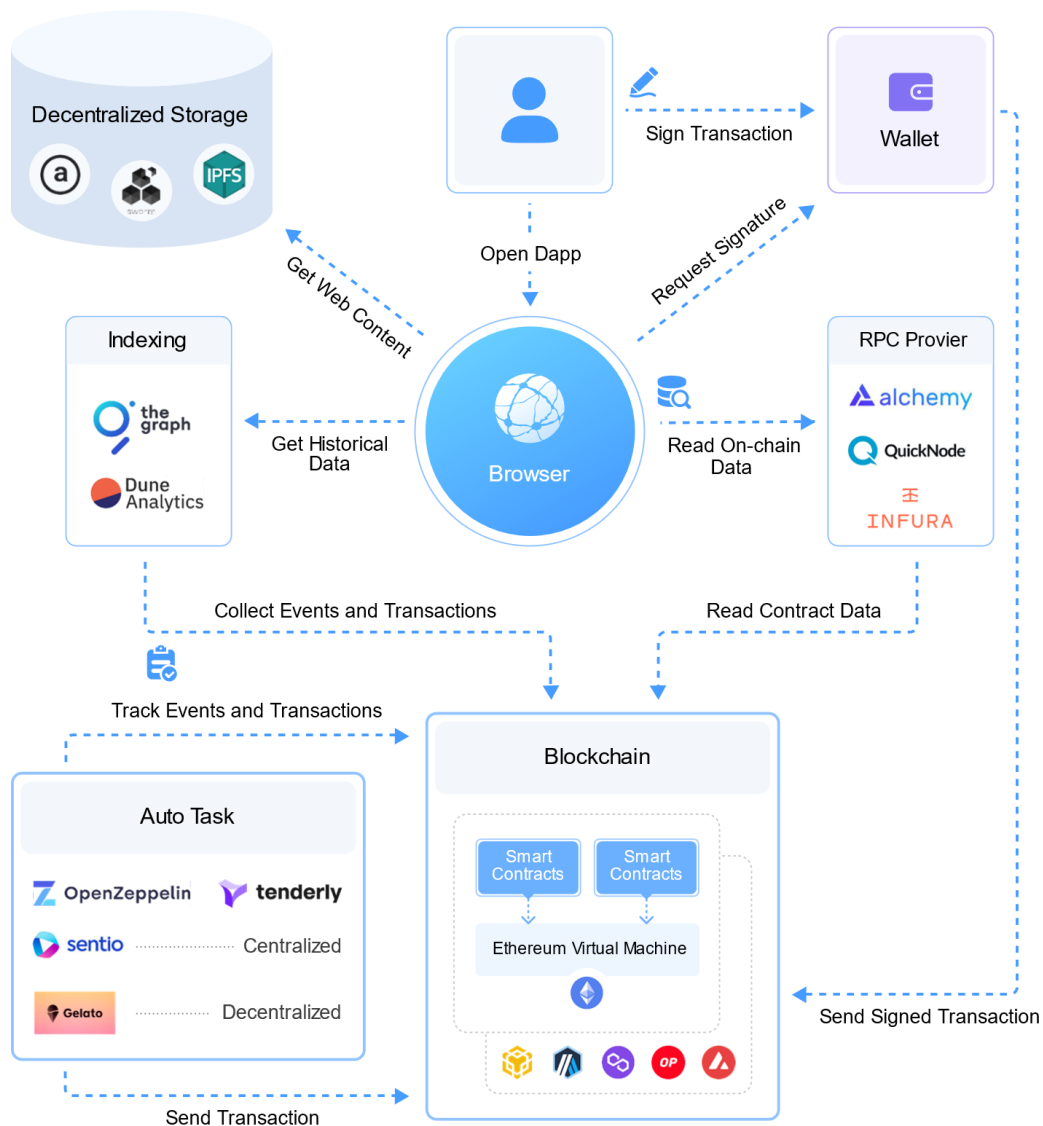
ЗМІСТ

ВСТУП.....	3
1 Аналіз сучасних стандартів безпеки вебдодатків (OWASP Top 10:2025)	5
1.1. Еволюція загроз: від OWASP 2021 до OWASP 2025	5
1.2. Детальна характеристика критичних вразливостей	6
1.3. Методологія оцінки ризиків та рівні критичності за стандартами OWASP7	
2 Архітектурні особливості та різноманітність децентралізованих систем	11
2.1. Загальна структура dApp: рівень протоколу, смарт-контрактів та клієнтського інтерфейсу.....	11
2.2. Огляд існуючих систем децентралізованих додатків.....	11
2.3. Роль оракулів та мостів (bridges) у функціонуванні екосистеми.....	14
3 Складання аналогічних вимог безпеки для децентралізованих додатків (Mapping).....	17
3.1. Адаптація OWASP Top 10 до специфіки блокчейну.....	17
3.2. Специфічні блокчейн-ризики поза межами традиційного OWASP	18
3.3. Аналіз ризиків ланцюжка постачання (A03:2025) у контексті бібліотек .	19
4 Розробка комплексних вимог до безпечної реалізації dApps	21
4.1. Формування технічних вимог до розробки смарт-контрактів	21
4.2. Вимоги до безпеки взаємодії з гаманцями та фронтенд-частиною	21
4.3. Рекомендації щодо моніторингу та реагування на інциденти	23
ВИСНОВКИ.....	25

ВСТУП

Зараз ми спостерігаємо, як парадигма розробки софту впевнено дрейфує в бік децентралізації. Головну роль тут грають dApps (Decentralized Applications). Якщо в класичній архітектурі «клієнт-сервер» ми довіряємо конкретній організації чи органу, то в децентралізованих додатках усе тримається на прозорості блокчейну та автономності смарт-контрактів. Але є нюанс. Принцип «Code is Law» (код – це закон) створює серйозні виклики для безпеки: будь-яка помилка в логіці стає публічною і, що важливо, незворотною. Адже блокчейн імутабельний, і з цим доводиться рахуватися.

Чому це дослідження актуальне? Річ у тім, що більшість зломів у Web3-екосистемі трапляються не через вразливості самих алгоритмів консенсусу, а через «дірки» на рівні додатків та їхньої взаємодії зі звичними вебтехнологіями. Оскільки dApp – це такий собі гібрид фронтенду (Web2) та смарт-контрактів (Web3), системному аналітику потрібен справді комплексний інструментарій для оцінки ризиків.



Методологія OWASP (Open Web Application Security Project) допомагає структурувати цей аналіз, спираючись на світовий досвід. Примітно, що оновлений стандарт OWASP Top 10:2025 пропонує нове бачення критичних загроз. Фокус змістився: тепер ми дивимося не стільки на поодинокі помилки в

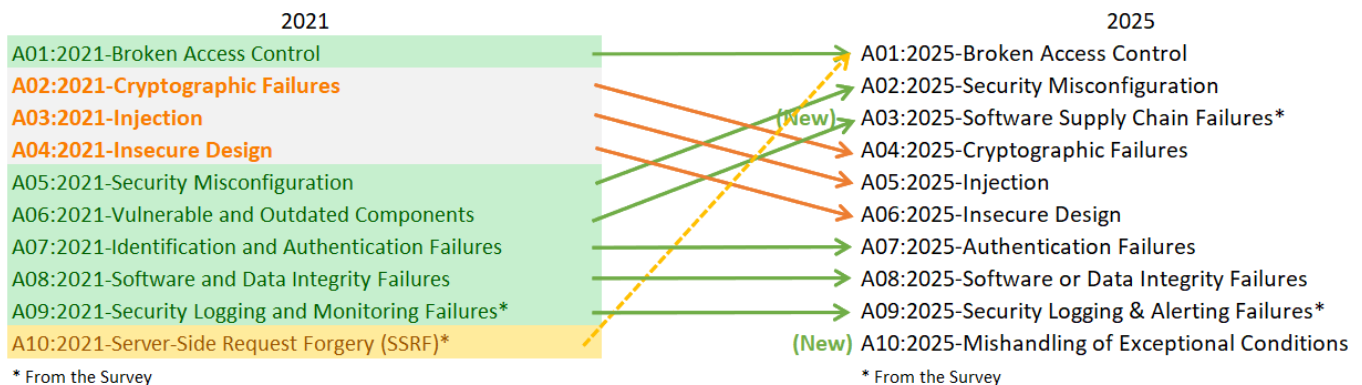
кодi, скiльки на системнi архiтектурнi дефекти та проблеми цiлiсностi даних. Власне, мета цiєї роботи – не просто переказати теорiю, а сформуванi практичнi вимоги до безпечного життєвого циклу розробки (S-SDLC). Це критично для захисту активiв користувачiв у досить ворожому середовищi публiчних мереж.

1 АНАЛІЗ СУЧАСНИХ СТАНДАРТІВ БЕЗПЕКИ ВЕБДОДАТКІВ (OWASP TOP 10:2025)

Проект OWASP давно став визнаним стандартом у кібербезпеці. Він дає розробникам та аналітиками об'єктивну базу для оцінки ризиків. Версія 2025 року – це свого роду відповідь на ускладнення сучасних стеків розробки. Інтеграція сторонніх сервісів та заплутана бізнес-логіка стали основними джерелами небезпеки.

1.1. Еволюція загроз: від OWASP 2021 до OWASP 2025

Якщо порівняти рейтинги 2021 та 2025 років, трансформація ландшафту загроз вражає. Раніше акцент часто робили на технічних методах обходу фільтрації (як-от Injection). Тепер же на перше місце виходять проблеми управління доступом, конфігурації та ланцюжків постачання. Це відображає нову реальність: хакери частіше експлуатують логічні прорахунки архітекторів, ніж дрібні помилки в синтаксисі, які припустив програміст.



Що варто зауважити в новому списку:

- **Домінування A01:2025 - Broken Access Control.** Ця категорія впевнено тримає лідерство. У світі, де API та мікросервіси спілкуються між собою автономно, будь-яке неправильне розмежування прав – це найкоротший шлях до витоку даних.
- **Стрімкий зліт A03:2025 - Software Supply Chain Failures.** Мабуть, це одна з найважливіших змін саме для dApp-систем. Сучасні смарт-контракти та вебпакети тягнуть за собою десятки залежностей. Якщо скомпрометовано бодай одну бібліотеку, яку ви імпортували просто «для зручності», уся система може отримати бекдор.
- **Акцент на системності через A06:2025 - Insecure Design.** OWASP нагадує нам: безпеку неможливо просто «наліпити» зверху на готовий продукт. Якщо архітектура від початку «крива» і не передбачає захисту, жодні патчі коду ситуацію не врятують.
- **Виокремлення A10:2025 - Mishandling of Exceptional Conditions.** Раніше на помилки обробки винятків часто заплющували очі. Тепер це серйозний ризик. Чому? Бо некоректний вихід системи зі штатного режиму часто відкриває вікно для несанкціонованих дій.

Для децентралізованих додатків цей список максимально релевантний. Скажімо, категорія A04:2025 (Cryptographic Failures) прямо стосується безпеки приватних ключів. А A09:2025 підсвічує проблему відсутності моніторингу в блокчейн-мережах, через що зупинити атаку на протокол вчасно буває майже неможливо.

Системний аналіз за цим стандартом допомагає зрозуміти не тільки те, де програма зламається, а й як вона поводить себе в стресових умовах. Для dApps, які працюють з фінансами, це життєво необхідно.

1.2. Детальна характеристика критичних вразливостей

Для системного аналітика глибоке розуміння кожної категорії OWASP Top 10:2025 – це фундамент моделі загроз. У цій версії ми бачимо остаточний перехід від аналізу суто програмних «багів» до аналізу системних дефектів. Нижче наведено технічний огляд усіх десяти категорій:

- **A01:2025 – Broken Access Control:** Коли обмеження на дії користувачів працюють абияк. Зловмисник отримує доступ до чужих функцій чи даних, може змінювати права інших або дивитися конфіденційні записи.
- **A02:2025 – Security Misconfiguration:** Банальні помилки в налаштуваннях. Стандартні паролі, відкриті порти, зайві увімкнені функції або «діряві» хмарні сховища.
- **A03:2025 – Software Supply Chain Failures:** Головний біль 2025 року. Використання чужих бібліотек чи API без перевірки цілісності. Зламали один модуль в NPM – зламали весь ваш додаток.
- **A04:2025 – Cryptographic Failures:** Раніше ми знали це як "Sensitive Data Exposure". Це слабе шифрування, «зашиті» прямо в код ключі або передача даних через HTTP.
- **A05:2025 – Injection:** Сюди відносимо все, де неперевірені дані потрапляють до інтерпретатора: від SQL-ін'єкцій до атак на рівні ORM. Рейтинг впав, але небезпека нікуди не зникла.
- **A06:2025 – Insecure Design:** Проблеми в самій логіці проектування. Якщо процес відновлення пароля не передбачає перевірки особи, це структурний дефект, який кодом не «залатаєш».
- **A07:2025 – Authentication Failures:** Проблеми з ідентифікацією. Без MFA або з поганою політикою паролів хакерам значно простіше використовувати брутфорс чи викрадені сесії.
- **A08:2025 – Software or Data Integrity Failures:** Коли ми сліпо довіряємо цілісності коду чи даних. Наприклад, приймаємо оновлення софту з неперевірених джерел, що дозволяє виконати будь-який шкідливий код.
- **A09:2025 – Security Logging and Alerting Failures:** Якщо критичні події не логуються, ви не дізнаєтеся про атаку вчасно. І, що гірше, не зможете провести розслідування після інциденту.
- **A10:2025 – Mishandling of Exceptional Conditions:** Вразливість вилазить, коли додаток «падає» чи видає помилку. Витік технічних подробиць (stack traces) часто дає хакеру підказку, як діяти далі.

Взаємозв'язок категорій OWASP Top 10:2025

Первинна вразливість	Вторинний наслідок	Опис механізму зв'язку
A03: Software Supply Chain Failures	A01: Broken Access Control	Компрометована бібліотека (напр. OpenZeppelin) може додати прихованого адміністратора ("backdoor admin").
A06: Insecure Design	A07: Authentication Failures	Відсутність перевірки особи у процесі відновлення пароля (дефект логіки) призводить до легкого брутфорсу або викрадення сесій.
A02: Security Misconfiguration	A10: Mishandling of Exceptional Conditions	Зайві увімкнені функції діагностики (A02) призводять до витоку stack traces при помилках, даючи хакеру підказки (A10).
A04: Cryptographic Failures	A08: Software or Data Integrity Failures	Використання слабких підписів (A04) дозволяє зловмиснику підробити оновлення контракту або метадані (A08).
A05: Injection	A06: Insecure Design	Маніпуляція вхідними параметрами через Calldata Injection стає можливою через відсутність лімітів та перевірок на рівні дизайну.

Цей аналіз підтверджує, що для децентралізованих додатків (dApps) безпека є комплексною: пробій у сторонньому SDK (A03) миттєво нівелює всі налаштування контролю доступу (A01) на рівні смарт-контракту.

1.3. Методологія оцінки ризиків та рівні критичності за стандартами OWASP

Щоб аналіз децентралізованих систем був справді об'єктивним, аналітику не обійтися без надійної математичної моделі. OWASP пропонує досить логічний підхід: ми оцінюємо ризик, виходячи з того, наскільки ймовірним є інцидент і наскільки боляче він вдарить по системі.

Основна формула для розрахунку виглядає так:

$$Risk = Likelihood \times Impact$$

Просто і дієво. Проте, диявол криється в деталях, а саме у факторах, що формують ці показники.

Чинники ймовірності (Likelihood):

- **Профіль агента загрози:** наскільки досвідченим є хакер, що його мотивує і які ресурси в нього під рукою.
- **Параметри вразливості:** чи легко знайти «дірку», наскільки важко її експлуатувати та чи знає про неї широка спільнота.

Фактори впливу (Impact):

- **Технічний:** чи призведе це до крадіжки даних, зміни коду або повної зупинки сервісу.
- **Бізнес-вплив:** тут і прямі фінансові втрати, і репутаційні ризики, і навіть юридичні «привіти» у вигляді штрафів GDPR чи кримінальних справ.

Щоб структурувати наш аналіз у цій лабораторній роботі, ми використаємо матрицю оцінки для всіх десяти категорій OWASP 2025. Це допоможе нам зрозуміти, за що хапатися в першу чергу.

Пріоритетність категорій за рівнями верифікації (ASVS)

Категорія OWASP 2025	Рівень критичності	Пріоритет виправлення
A01: Broken Access Control	Critical	Найвищий (негайне втручання)
A02: Security Misconfiguration	High	Високий (налаштування середовища)
A03: Software Supply Chain Failures	Critical	Найвищий (перевірка залежностей)
A04: Cryptographic Failures	High	Високий (захист даних)
A05: Injection	Medium/High	Середній (фільтрація вводу)
A06: Insecure Design	High	Високий (перегляд архітектури)
A07: Authentication Failures	High	Високий (контроль доступу)

A08: Software or Data Integrity Failures	Medium	Середній (контроль цілісності)
A09: Security Logging and Alerting Failures	Medium	Середній (налаштування моніторингу)
A10: Mishandling of Exceptional Conditions	Low/Medium	Низький (обробка помилок)

Варто зауважити, що рівні верифікації безпеки (Level 1, 2, 3) за стандартом OWASP ASVS визначають, наскільки глибоко ми копаємо під час аудиту. Примітно, що для децентралізованих систем, які керують фінансовими активами, Level 3 – це не побажання, а необхідність. Він вимагає повного розбору коду, формальної верифікації логіки та жорстких стрес-тестів. Тільки так можна отримати хоч якісь гарантії стійкості у ворожому середовищі блокчейну.

Економіка безпеки: Витрати на виправлення вразливостей

Ця таблиця демонструє, чому принцип "Security by Design" є не просто гаслом, а фінансовою необхідністю. Витрати вказані умовними одиницями (коефіцієнт зростання вартості).

Категорія OWASP 2025	Design (Проектування)	Dev (Розробка)	Test (Тестування)	Production (Після деплою)
A01: Access Control	1x	5x	20x	100x+ (Ризик втрати TVL)
A02: Misconfiguration	1x	2x	10x	50x
A03: Supply Chain	1x	10x	40x	150x (Повна заміна архітектури)
A04: Cryptographic	1x	15x	50x	200x (Неможливість відкату)
A05: Injection	1x	3x	15x	60x

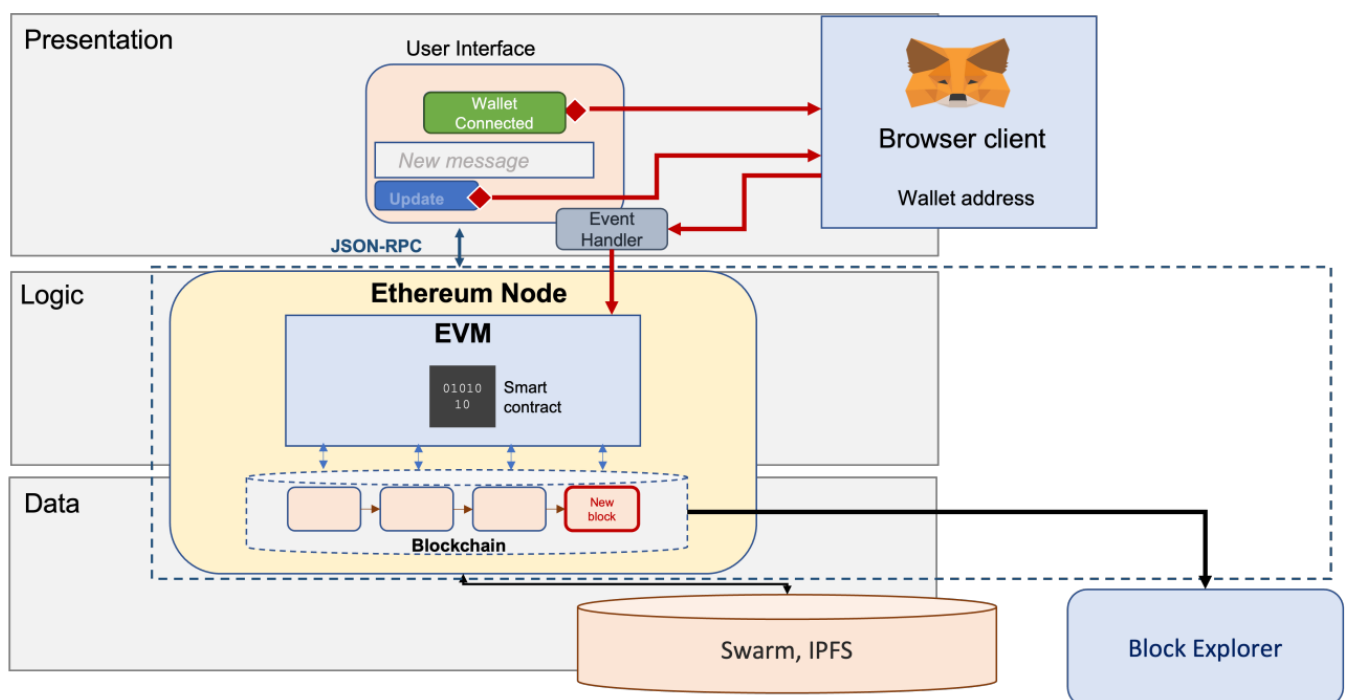
A06: Insecure Design	1x	20x	80x	300x (Повний редизайн)
A07: Authentication	1x	5x	15x	80x
A08: Integrity	1x	10x	30x	120x
A09: Logging	1x	2x	5x	20x
A10: Exceptional Cond.	1x	4x	12x	70x

2 АРХІТЕКТУРНІ ОСОБЛИВОСТІ ТА РІЗНОМАНІТНІСТЬ ДЕЦЕНТРАЛІЗОВАНИХ СИСТЕМ

2.1. Загальна структура dApp: рівень протоколу, смарт-контрактів та клієнтського інтерфейсу

Якісний аудит безпеки починається з розуміння того, як усе влаштовано. Системний аналітик має розрізняти три основні рівні архітектури dApp, адже кожен із них – це окрема точка входу для загроз. Сучасний dApp – це химерний симбіоз Web2 та Web3, що, природно, розширює поверхню атаки.

1. **Рівень протоколу та мережі (Blockchain/L1/L2):** Це фундамент системи. Він тримає на собі консенсус і валідацію транзакцій. Хоча цей рівень зазвичай вважають найбільш захищеним, він вразливий до помилок конфігурації (A02:2025) на рівні вузлів. Що важливо, моніторинг аномалій тут (A09:2025) часто перетворюється на квест через саму природу децентралізації.
2. **Рівень смарт-контрактів (Business Logic Layer):** Справжнє серце додатка, де «живе» код на Solidity або Rust. Саме тут зосереджені найбільші ризики. Будь-яка логічна помилка миттєво стає критичною вразливістю доступу (A01:2025) або архітектурним дефектом (A06:2025), якщо хтось зможе зманіпулювати ліквідністю. На цьому рівні критично важливо правильно перевіряти підписи та генерувати випадкові числа (A04:2025).
3. **Рівень клієнтського інтерфейсу та провайдерів (Frontend & Middleware):** Те, що бачить користувач у своєму браузері чи гаманці (MetaMask, Phantom). Цей рівень найбільше страждає від вразливостей ланцюжка постачання (A03:2025) через використання тисяч npm-пакетів. Також тут часто вилазять проблеми з автентифікацією (A07:2025), коли додаток плутається у статусах підключення гаманця.



2.2. Огляд існуючих систем децентралізованих додатків

Погляньмо на ринок: він представлений різними екосистемами, і в кожній свій підхід до безпеки. Чому аналітику важливо знати ці відмінності? Бо специфіка віртуальної машини (VM) безпосередньо впливає на те, як саме проявляються вразливості OWASP 2025.

- **EVM-сумісні системи (Ethereum, BNB Chain, Polygon):** Використовують віртуальну машину Ethereum. Це зріла екосистема з купою стандартів. Проте через специфічну стекову модель виконання вона вразлива до атак на цілісність (A08:2025). Часто розробники просто копіюють чужий код (форки), бездумно переносячи параметри безпеки одного ланцюжка в інший (A02:2025), що призводить до катастроф.
- **Non-EVM системи (Solana, Aptos, Near):** Використовують зовсім інші моделі, наприклад, Account-based у Solana. Тут код виконується паралельно, тому контроль за станами (A10:2025) має бути надзвичайно суворим. Неправильно оброблений виняток – і ви отримуєте критичну помилку доступу.

Ось як ці вразливості виглядають «у дикій природі»:

Категорія OWASP 2025	Прояв у EVM (напр. Ethereum)	Прояв у Non-EVM (напр. Solana)
A01: Broken Access Control	Помилки в модифікаторах onlyOwner.	Проблеми перевірки власника аккаунту.
A02: Security Misconfiguration	Криві параметри Gas Limit або Proxy.	Помилки конфігурації Program ID.
A03: Supply Chain Failures	Вразливі версії бібліотек OpenZeppelin.	Компрометація SDK (напр. Solana Web3.js).
A04: Cryptographic Failures	Використання передбачуваного block.timestamp.	Слабкі криві в кастомних підписах.
A05: Injection	SQL-ін'єкції в індексаторах (The Graph).	Command Injection у скриптах розгортання.
A06: Insecure Design	Відсутність лімітів на Flash Loans.	Помилки логіки в голосуванні DAO.

A07: Authentication Failures	Фішинг через підпис eth_sign.	Вкрадені Seed-фрази через розширення.
A08: Integrity Failures	Несанкціоноване оновлення через Proxy.	Підміна метаданих у програмах.
A09: Logging/Alerting Failures	Відсутність Event-логів для дій.	Ігнорування помилок у транзакціях.
A10: Exceptional Conditions	Некоректний revert при викликах.	Паніка (Panic) через переповнення стеку.

Статистика зломів за типами вразливостей (Ethereum vs Solana vs BSC)

Кожна мережа має свої "слабкі місця" через особливості архітектури віртуальних машин.

Категорія OWASP 2025	Ethereum (EVM)	Solana (Non-EVM)	BSC (EVM Fork)
Домінуючий ризик	A06 (Економічні атаки)	A01/A10 (Логіка акаунтів)	A03 (Вразливі форки)
Типова атака	Reentrancy, Flash Loans	Account Substitution	Rug pulls через вразливі SDK
Частота інцидентів	Середня (висока вартість атак)	Висока (складність Rust-логики)	Дуже висока (копіювання коду)
Критична точка	Смарт-контракти (Solidity)	Runtime (Sealevel)	Конфігурація вузлів та проксі

Аналіз ТОП-3 dApp-протоколів через призму OWASP

Навіть гіганти ринку змушені будувати багаторівневий захист.

Протокол	Ключові категорії захисту	Технічні заходи безпеки

Uniswap	A06, A08	Імутабельність контрактів, відсутність адмін-ключів, захист від маніпуляцій оракулами.
Aave	A01, A06	Складна система Governance, ліміти на Flash Loans, багаторівневі перевірки ліквідності.
Lido	A01, A04	Мультисіг-управління (t-of-n), аудовані оракули для стейкінгу, формальна верифікація коду.

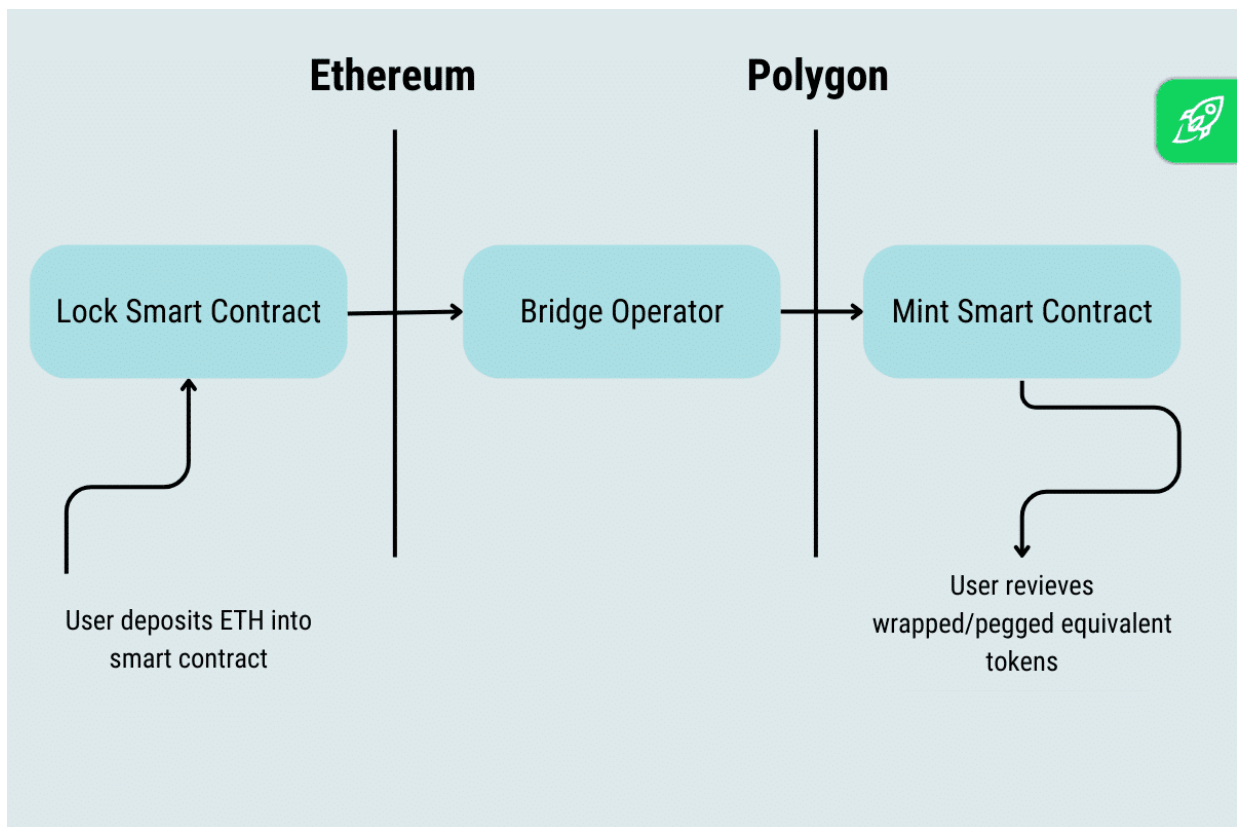
Підсумовуючи, хоча технології можуть різнитися, принципи OWASP 2025 залишаються універсальним мірилом. Важливо пам'ятати: захист dApp – це не лише про ідеальний смарт-контракт, а про цілісну перевірку всіх рівнів системи.

2.3. Роль оракулів та мостів (bridges) у функціонуванні екосистеми

Під час системного аналізу децентралізованих додатків ми просто не можемо ігнорувати компоненти, які зв'язують ізольований блокчейн із реальним світом чи іншими мережами. Оракули та крос-чейн мости – це свого роду інфраструктурний «клей». Проте варто зауважити: якщо цей клей реалізовано криво, він миттєво перетворюється на критичну точку відмови, що тягне за собою ледь не весь перелік ризиків OWASP Top 10:2025.

Оракули – це сервіси, які «годують» смарт-контракт зовнішніми даними: цінами активів, результатами матчів чи прогнозом погоди. Головна біда тут у довірі. Якщо додаток сліпо вірить лише одному оракулу (наприклад, бере ціну з однієї конкретної DEX), він стає вразливим до маніпуляцій. Чи можна назвати такий дизайн безпечним? Навряд чи. У термінах OWASP це класика: **A06:2025 (Insecure Design)** та **A08:2025 (Software or Data Integrity Failures)**, оскільки цілісність інформації на вході не підкріплена децентралізованим консенсусом.

Крос-чейн мости (Bridges) – конструкції ще складніші. Вони потрібні, щоб перекидати активи між різними мережами, наприклад, з Ethereum на Polygon. Зазвичай вони тримаються на смарт-контрактах в обох мережах та валідаторах (релеєрах). Мости зараз – це ціль номер один для атак типу **A01:2025 (Broken Access Control)**, коли хакери намагаються підробити підпис валідатора. Також часто виникає **A10:2025 (Mishandling of Exceptional Conditions)**: помилка в одній мережі може призвести до подвійного витрачання або просто до того, що ваші кошти назавжди «зависнуть» в іншій.



Ось як виглядає аналіз ризиків цих компонентів за списком OWASP 2025:

Категорія OWASP 2025	Ризики для Оракулів та Мостів
A01: Broken Access Control	Хтось чужий оновлює список валідаторів мосту.
A02: Security Misconfiguration	Помилки в налаштуваннях порогу підписів (наприклад, t-of-n).
A03: Supply Chain Failures	Вразливі SDK для зв'язку з API бірж.
A04: Cryptographic Failures	Слабкі параметри ECDSA або інші застарілі схеми підпису.
A05: Injection	Шкідливий код у метаданих повідомлень мосту.
A06: Insecure Design	Цінові оракули без захисту від "flash loan" атак.
A07: Authentication Failures	Вкрадені приватні ключі адміністраторів чи валідаторів.

A08: Integrity Failures	Підміна даних опракула (атака Man-in-the-middle).
A09: Logging/Alerting Failures	Мовчання системи при аномальних обсягах переказів.
A10: Exceptional Conditions	Транзакція «зависає», бо мережа перевантажена.

Примітно, що для системного аналітика безпека цих вузлів є пріоритетом. Їхній злом – це завжди катастрофа для всієї екосистеми.

3 СКЛАДАННЯ АНАЛОГІЧНИХ ВИМОГ БЕЗПЕКИ ДЛЯ ДЕЦЕНТРАЛІЗОВАНИХ ДОДАТКІВ (MAPPING)

3.1. Адаптація OWASP Top 10 до специфіки блокчейну

Цей етап у нашій лабораторній роботі є ключовим. Він показує, чи вміє аналітик адаптувати загальні правила веббезпеки до специфічного «дикого заходу» блокчейну. Тут з'являються нові змінні: газ, ліміти транзакцій, публічність стану та незворотність будь-якої дії.

Ось як ми трансформуємо (Mapping) категорії OWASP 2025 для потреб dApps:

- **A01: Broken Access Control => Помилки логіки прав доступу.** У блокчейні все зав'язано на модифікаторах. Забули додати `require(msg.sender == owner)` у функцію `withdraw()`? Будь-хто зможе спустошити контракт. Сюди ж відносимо і маніпуляції з правами голосу в DAO.
- **A03: Software Supply Chain Failures => Безпека сторонніх контрактів.** Ми часто успадковуємо готові рішення, наприклад, ERC20 від OpenZeppelin. Але якщо в базовій бібліотеці чи скриптах розгортання є «дірка», ваш додаток отримає її у спадок. Аналітик повинен перевіряти кожну залежність. Кожну.
- **A05: Injection => Небезпечні параметри виклику (Calldata Injection).** Класичні SQL-ін'єкції тут – рідкість. Натомість зловмисники маніпулюють вхідними параметрами, щоб підмінити адресу цільового контракту прямо в процесі виклику.
- **A10: Mishandling of Exceptional Conditions => Вразливості "DOS with Revert".** Якщо смарт-контракт не знає, що робити, коли зовнішній виклик повертає помилку, транзакція може просто заблокуватися. Це ідеальний інструмент для атак на аукціони чи системи виплат.

Категорія OWASP 2025	Проекція на блокчейн (Web3)	Приклад атаки
A01: Access Control	Функції без обмежень доступу.	Виклик <code>initContract()</code> уже після запуску.
A02: Misconfiguration	Проблеми з Проху-контрактами.	Злам логіки через підступний <code>delegatecall</code> .
A03: Supply Chain	«Діряві» пакети в NPM/Truffle/Hardhat.	Шкідливий код у плагіні для деплою.

A04: Cryptographic	Погані джерела випадковості.	Вгадування результатів лотереї через blockhash.
A05: Injection	Маніпуляції з ABI encoding.	Переповнення буфера при парсингу даних.
A06: Insecure Design	Економічні діри (Flash Loans).	Маніпуляція ціною для примусової ліквідації.
A07: Authentication	Слабкий захист приватних ключів.	Викрадення ключа адміністратора.
A08: Integrity	Відсутність перевірки стану перед дією.	Reentrancy (повторний вхід у функцію).
A09: Logging/Alerting	Відсутність емісії подій (Events).	Тихе виведення коштів без запису в лог.
A10: Exceptional	Помилки при перевірці require/assert.	Блокування виплат через невдалий зовнішній виклик.

Використовуючи цей "Mapping", системний аналітик створює чіткий чек-лист. Це дозволяє ще на етапі проектування відсікти цілі класи вразливостей.

3.2. Специфічні блокчейн-ризики поза межами традиційного OWASP

Стандарт OWASP Top 10:2025 – це, примітно, чудовий фундамент для аналізу. Проте чи вичерпний він для специфічного світу блокчейну? На перший погляд, так, але існують класи вразливостей, притаманні суто архітектурі смарт-контрактів. Системному аналітику важливо вплітати ці «Web3-оригінальні» загрози в загальну модель безпеки, адже вони часто стають ідеальним інструментом для експлуатації ризиків із категорій A06:2025 (Insecure Design) та A08:2025 (Integrity Failures).

1. **Reentrancy (Повторний вхід):** Класика жанру. Атака виникає, коли контракт звертається до іншого (зловмисного) контракту ще до того, як оновив свій внутрішній стан – наприклад, баланс. Це дозволяє хакеру рекурсивно викачувати кошти, доки вони не закінчаться.
2. **MEV (Maximal Extractable Value) та Front-running:** Оскільки все в мемпулі публічне, валідатори або боти можуть спритно маніпулювати порядком транзакцій. Скажімо, «сендвіч-атака» дозволяє отримати прибуток за рахунок проковзування (slippage) звичайного користувача на біржі.

3. **Flash Loan Attacks (Атаки з миттєвими позиками):** Можливість взяти величезну суму без застави в межах однієї транзакції розв'язує руки зловмисникам. Вони маніпулюють оракулами або просто захоплюють управління (Governance) в DAO.
4. **Gas Limit & Denial of Service:** Специфічний вид DoS. Тут зловмисник змушує контракт виконувати такі важкі операції, що вони просто не влязять у ліміт газу блоку. Контракт «зависає».

Варто зауважити, як традиційні категорії OWASP 2025 переплітаються з цими специфічними ризиками:

Категорія OWASP 2025	Специфічний блокчейн-ризик (Web3 Nuance)	Вплив на систему
A01: Broken Access Control	Маніпуляція голосуванням через Flash Loans.	Захоплення контролю над казначейством.
A02: Security Misconfiguration	Помилки у виставленні Gas Limit у проксі.	Відмова в обслуговуванні при оновленні.
A03: Supply Chain Failures	Використання неперевіраних копій (Clones).	Успадкування прихованих вразливостей Reentrancy.
A04: Cryptographic Failures	Передбачувана випадковість (block.prevrandao).	Маніпуляція результатами розіграшів.
A05: Injection	Логічна ін'єкція через невідповідність типів.	Зміна балансів (у старих версіях мов).
A06: Insecure Design	Економічні атаки на пули ліквідності.	Каскадне ліквідування позицій користувачів.
A07: Authentication	Слабке управління мультисігами (Gnosis Safe).	Блокування коштів через втрату доступу підписантів.
A08: Integrity Failures	Атаки типу Transaction Ordering (Front-running).	Крадіжка прибутку через маніпуляцію чергою.
A09: Logging/Alerting	Відсутність індексації подій для MEV-ботів.	Неможливість виявити атаку до фіналізації блоку.
A10: Exceptional Conditions	Необробка помилки call.value у Solidity.	Назавжди заблоковані кошти ("Frozen funds").

3.3. Аналіз ризиків ланцюжка постачання (A03:2025) у контексті бібліотек

Якщо ми говоримо про dApps, то категорія A03:2025 (Supply Chain Failures) стає справжнім викликом. Web3 тримається на принципі компонованості (composability). Ми ж майже нічого не пишемо з нуля, чи не так? Стандартні бібліотеки на кшталт OpenZeppelin та інструменти на кшталт Hardhat чи Foundry – це база, на якій все стоїть.

Головні вектори атак на Supply Chain тут такі:

- **Компрометація бібліотек (Library Injection):** Якщо хакер дістанеться популярного репозиторію на GitHub, шкідливий код розлетиться по тисячах контрактів ще під час компіляції. Це може бути що завгодно: від «чорного ходу» для виведення грошей до зміни логіки прав доступу.
- **Ризики Proxy та Upgradeability:** Багато dApps оновлюються через проксі. Якщо скрипт оновлення підв'язаний до зовнішнього джерела, яке зламали, вся ваша система миттєво заміниться на шкідливу версію.
- **Неперевірений вихідний код (Unverified Source Code):** Ми звикли довіряти тому, що бачимо на Etherscan. Проте, якщо процес деплою непрозорий, байткод у блокчейні може зовсім не відповідати тому «красивому» коду, який опубліковано.

Важливо оцінювати ці ризики для кожної категорії OWASP 2025:

Категорія OWASP 2025	Вплив Supply Chain ризику (A03) на категорію
A01: Access Control	Шкідлива бібліотека може додати «таємного» адміна.
A02: Misconfiguration	Скрипти деплою встановлюють небезпечні дефолтні налаштування.
A03: Supply Chain Failures	Пряма компрометація залежностей.
A04: Cryptographic Failures	Використання бібліотек із вразливими zk-SNARKs.
A05: Injection	Впровадження шкідливих фрагментів через макроси компілятора.
A06: Insecure Design	Успадкування архітектурних дірок від популярних шаблонів.
A07: Authentication	Використання зламаних SDK гаманців (WalletConnect).
A08: Integrity Failures	Підміна версії компілятора (Compiler Trojan).
A09: Logging/Alerting	Видалення функцій логування в «лівій» версії бібліотеки.
A10: Exceptional Conditions	Зміна логіки помилок для приховування крадіжки газу.

Щоб спати спокійніше, системний аналітик має впроваджувати практику SBOM (Software Bill of Materials). Це дозволяє точно знати «склад» кожного смарт-контракту і вчасно бачити вразливі компоненти ще до розгортання.

4 РОЗРОБКА КОМПЛЕКСНИХ ВИМОГ ДО БЕЗПЕЧНОЇ РЕАЛІЗАЦІЇ DAPPS

Підсумовуючи аналіз OWASP та специфічних Web3-загроз, ми можемо сформулювати чіткі вимоги. Вони стануть фундаментом для ТЗ та головними критеріями під час майбутніх аудитів.

4.1. Формування технічних вимог до розробки смарт-контрактів

Код у блокчейні живе в публічному полі, де будь-яка помилка миттєво стає здобиччю ботів. Саме тому вимоги мають бути не просто суворими, а вичерпними – від першого рядка коду до моменту деплою.

Основні технічні вимоги:

1. **A01: Access Control:** Тільки перевірені рішення для ролей (наприклад, AccessControl від OpenZeppelin). Будь-яка функція, що змінює стан, повинна мати свій модифікатор доступу.
2. **A02: Misconfiguration:** Жодних «магічних чисел» чи вшитих адрес. Усе критичне (адреси оракулів, ліміти) ініціалізується через конструктор із можливістю безпечного оновлення.
3. **A03: Supply Chain:** Тільки стабільні версії компілятора та верифіковані бібліотеки. Усі залежності фіксуються (pinned) за хеш-сумою.
4. **A04: Cryptographic:** Надійні джерела випадковості (на кшталт Chainlink VRF). Забудьте про параметри блоку для лотерей.
5. **A05: Injection:** Суворі типізація. У старих версіях Solidity (<0.8.0) – тільки бібліотеки для безпечної арифметики. Валідація байтових масивів обов'язкова.
6. **A06: Insecure Design:** Впровадження «тривожної кнопки» (Circuit Breaker). Потрібна можливість зупинити функції, якщо щось пішло не так.
7. **A07: Authentication:** Захист від Signature Replay Attack за допомогою nonce та chainId.
8. **A08: Integrity:** Захист від Reentrancy через патерн Checks-Effects-Interactions або модифікатор nonReentrant. Ніякого виконання довільного коду через call без фільтрів.
9. **A09: Logging:** Кожна критична дія – від зміни власника до виведення коштів – ПОВИННА генерувати подію (Event). Без цього моніторинг неможливий.
10. **A10: Exceptional Conditions:** Перевірка результатів усіх зовнішніх викликів. Кожен call чи transfer має супроводжуватися логікою require(success).

Використовуючи цей чек-лист, системний аналітик може ще на старті відсікти цілі класи вразливостей, характерних для децентралізованих систем.

4.2. Вимоги до безпеки взаємодії з гаманцями та фронтенд-частиною

Фронтенд у dApp – це, власне, місток між користувачем і блокчейном. Але примітно, що саме він найчастіше стає головною мішенню для фішингу чи підступної підміни транзакцій. Вимоги до інтерфейсу мають бути прописані так, щоб людина завжди бачила реальну картину того, що вона підписує.

Варто зауважити, як вимоги клієнтського рівня лягають на сітку OWASP 2025:

1. **A01: Access Control:** Клієнтська сторона має сама перевіряти статус гаманця та Chain ID перед кожним кроком. Це банальний, але дієвий спосіб не дати транзакції полетіти «не в ту» мережу.
2. **A02: Misconfiguration:** Налаштування Content Security Policy (CSP) повинно бути максимально суворим. Тільки так можна закрити шлях шкідливим скриптам із лівих ресурсів.
3. **A03: Supply Chain:** Потрібно регулярно «проганяти» node_modules через сканери на кшталт npm audit або Snyk. І, що важливо, краще мінімізувати кількість зовнішніх Web3-бібліотек.
4. **A04: Cryptographic:** Про зберігання приватних ключів чи seed-фраз у LocalStorage або SessionStorage навіть мови бути не може. Це табу. Користуємося лише перевіреними Web3-провайдерами.
5. **A05: Injection:** Усе, що приходить із блокчейну (імена, описи NFT), треба ретельно фільтрувати. Інакше XSS не забариться.
6. **A06: Insecure Design:** Користувач має бачити чіткі попередження про ризики. Наприклад, коли підписує approve на безлімітну суму. Чи розуміє він наслідки?
7. **A07: Authentication:** Використовуємо EIP-712 для підпису повідомлень. Замість незрозумілого шістнадцяткового «сміття» людина має бачити структурований текст.
8. **A08: Integrity:** Потрібно захистити інтерфейс від підміни адреси в буфері обміну та обов'язково перевіряти адресу контракту безпосередньо перед викликом.
9. **A09: Logging:** Фронтенд має фіксувати підозрілі підключення гаманців або часті відмови від підпису. Це допоможе вчасно розгледіти початок фішингової атаки.
10. **A10: Exceptional Conditions:** Помилки блокчейну (той самий "Out of Gas") слід виводити у зрозумілій формі. Технічні тельбухи провайдера користувачеві знати ні до чого.

Матриця вимог до безпеки фронтенду dApp

Категорія OWASP 2025	Загроза для клієнта	Технічне рішення (Вимога)
A01: Access Control	Транзакція не в тій мережі	Валідація Chain ID та статусу гаманця перед викликом send().

A02: Misconfiguration	XSS через завантажені скрипти	Суворий Content Security Policy (CSP) та заборона inline скриптів.
A03: Supply Chain	Шкідливий код у npm-пакетах	Регулярний npm audit, фіксація версій (lock-files) та використання SBOM.
A04: Cryptographic	Крадіжка ключів з браузера	Повна заборона зберігання приватних ключів у LocalStorage/SessionStorage.
A05: Injection	XSS через дані з блокчейну	DOM Sanitization (напр. DOMPurify) для всіх даних, що приходять з контрактів.
A06: Insecure Design	Сліпе підписання (Blind Signing)	Візуалізація параметрів транзакції через EIP-712 (структуровані дані).
A07: Authentication	Фішинг-підписи	Перевірка origin запиту та використання nonce для захисту від Replay атак.
A08: Integrity	Підміна адреси в буфері	UI-попередження (checksums, іконки адрес) та звірка адреси перед самим підписом.
A09: Logging	Непомітний перебір гаманців	Логування підозрілих provider івентів та моніторинг аномальних відмов.
A10: Exceptional Cond.	Незрозумілі помилки ("Revert")	Мапінг технічних помилок EVM у зрозумілі UX-повідомлення (без витоку логів).

Системному аналітику варто наполягати на підході "Zero Trust": фронтенд не вірить блокчейну «на слово» без перевірки, а контракт, своєю чергою, не довіряє фронтенду. Тільки така двостороння валідація тримає систему в купі.

4.3. Рекомендації щодо моніторингу та реагування на інциденти

Коли ми запускаємо dApp у відкритому блокчейні, пасивна безпека перестає працювати. Потрібен активний моніторинг у реальному часі. Оскільки транзакції

публічні, боти-зловмисники знайдуть вразливість за секунди. Наше завдання – передбачити аномалії та мати готовий протокол «на випадок пожежі».

Головні акценти моніторингу через призму OWASP 2025:

- **A09: Security Logging and Alerting Failures:** На перший погляд, це серце моніторингу. Треба впровадити системи індексації (The Graph або свої воркери), щоб ловити кожен emit Event. Якщо хтось виводить величезну суму або змінює права доступу, а сповіщення мовчить – це провал.
- **A01: Access Control:** Стежимо за всіма викликами адмін-функцій. Будь-яка зміна власників має миттєво летіти в Slack чи PagerDuty.
- **A06: Insecure Design:** Моніторимо економіку. Різкі стрибки цін в оракулах чи раптові Flash Loans – це тривожний дзвіночок.
- **A08: Integrity:** Використовуємо Forta або Tenderly, щоб бачити підозрілу поведінку. Наприклад, якщо одну функцію смикають рекурсивно – це явна ознака Reentrancy.

Як ми будемо реагувати?

Для кожної категорії OWASP потрібен свій план.

1. **Автоматизовані зупинки (Circuit Breakers):** Якщо ми бачимо аномальний відтік грошей (A08) чи маніпуляції з цінами (A06), у контракті має спрацювати функція pause(). Її може активувати або бот-охоронець, або мультисіг.
2. **Механізми Upgradability:** Якщо вразливість знайшли в бібліотеках (A03) чи логіці (A04), треба мати змогу «накатати» виправлену версію через Проху. Але пам'ятаємо про Timelocks – користувачі мають бачити прозорість змін.

Категорія OWASP 2025	Метод виявлення (Monitoring)	Дія (Response)
A01: Access Control	Трейс викликів onlyOwner.	Блокування через Timelock.
A06: Insecure Design	Відстеження Slippage у пулах.	Встановлення лімітів на об'єм.
A08: Integrity	Детекція Reentrancy через стек викликів.	Активація Safe-mode (стоп виплат).
A10: Exceptional	Сплески помилок "Out of Gas".	Оптимізація лімітів газу.

ВИСНОВКИ

Під час роботи ми провели детальне дослідження безпеки dApps, спираючись на свіжий стандарт OWASP Top 10:2025. Примітно, що роль системного аналітика у Web3 стає фундаментальною: старі методи веб-безпеки тут потребують серйозної адаптації.

Що ми маємо в підсумку:

- **Адаптація стандартів:** Ми побачили, що хоча OWASP спочатку писали для звичайного вебу, всі 10 категорій ідеально лягають на блокчейн. Найбільш критичними точками виявилися контроль доступу (A01), ланцюжки постачання (A03) та архітектурні прорахунки (A06).
- **Аналіз екосистем:** Безпека dApp – це не лише код контракту. Це ціла система: фронтенд, оракули, мости. Пробій на будь-якому рівні руйнує все.
- **Формування вимог:** Тепер у нас є технічний перелік для розробників, що покриває весь шлях продукту – від перших креслень до моніторингу в «полі».

Підсумовуючи:

Децентралізація – це не синонім безпеки. Радше навпаки: оскільки код і активи публічні, аналітик має сповідувати "Zero Trust" і завжди бути на крок попереду. Методика Mapping OWASP to Web3 дозволяє побудувати реальний захист, де код і оперативне реагування працюють синхронно.

Таким чином, мета роботи досягнута: отримано навички проектування безпечних децентралізованих систем та оцінки ризиків згідно з актуальними світовими стандартами кібербезпеки 2025 року.